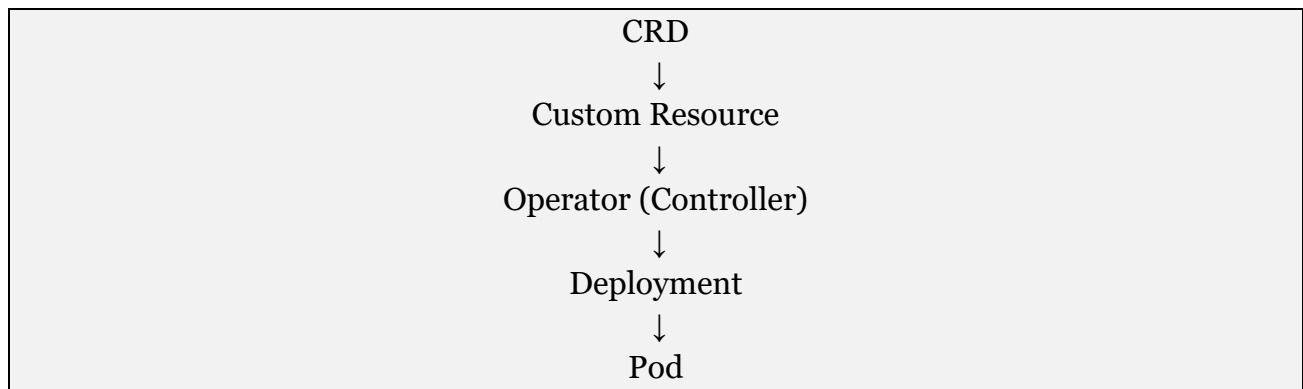


Lab Exercise 4- Build a Simple Kubernetes Operator using CRD + Custom Resource + Controller on Amazon EKS

Lab Objective

In this lab, you will learn the complete flow of:



You will create:

1. A Custom Resource Definition (CRD) named WebApplication
 2. A Custom Resource (CR) named ecommerce-app
 3. A simple Operator using Python
 4. Automatic Deployment creation from the CR
 5. Access the deployed NGINX application
-

Final Architecture

```
Developer
|
Apply WebApplication CR
|
WebApplication CR
|
Python Operator
|
+---- Creates Deployment
|
+---- Creates Service
|
NGINX Pods
|
LoadBalancer Service
|
Browser
```

Prerequisites

- Amazon EKS Cluster
- kubectl configured
- Python 3
- Docker
- Access to deploy pods

Verify cluster:

kubectl get nodes

Expected:

NAME	STATUS
ip-192-168-1-100.ec2.internal	Ready

Step 1: Create Working Directory

```
mkdir webapp-operator-lab  
cd webapp-operator-lab
```

Step 2: Create CRD

Create:

```
webapplication-crd.yaml
```

Paste:

```
apiVersion: apiextensions.k8s.io/v1  
kind: CustomResourceDefinition  
  
metadata:  
  name: webapplications.apps.company.com  
  
spec:  
  group: apps.company.com  
  
  names:  
    kind: WebApplication  
    plural: webapplications  
    singular: webapplication  
    shortNames:  
      - webapp  
  
  scope: Namespaced  
  
  versions:  
    - name: v1  
      served: true  
      storage: true  
  
  schema:  
    openAPIV3Schema:
```

```
type: object
```

```
properties:
```

```
spec:
```

```
type: object
```

```
properties:
```

```
image:
```

```
type: string
```

```
replicas:
```

```
type: integer
```

```
port:
```

```
type: integer
```

Deploy:

```
kubectl apply -f webapplication-crd.yaml
```

Verify:

```
kubectl get crds
```

Expected:

webapplications.apps.company.com

Step 3: Create Operator Service Account

Create:

```
operator-rbac.yaml
```

Paste:

```
apiVersion: v1
kind: ServiceAccount

metadata:
  name: webapp-operator

---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole

metadata:
  name: webapp-operator

rules:
- apiGroups: ["apps.company.com"]
  resources: ["webapplications"]
  verbs: ["*"]

- apiGroups: ["apps"]
  resources: ["deployments"]
  verbs: ["*"]

- apiGroups: [""]
  resources: ["services"]
  verbs: ["*"]

---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding

metadata:
  name: webapp-operator
```

```
subjects:
- kind: ServiceAccount
  name: webapp-operator
  namespace: default

roleRef:
  kind: ClusterRole
  name: webapp-operator
  apiGroup: rbac.authorization.k8s.io
```

Deploy:

```
kubectl apply -f operator-rbac.yaml
```

Step 4: Create Simple Python Operator

Create:

```
webapp_operator.py
```

Paste:

```
from kubernetes import client, config, watch
from kubernetes.client.rest import ApiException
import traceback
import time

print("Starting WebApplication Operator...")

# Load EKS/Kubernetes in-cluster config
config.load_incluster_config()

crd_api = client.CustomObjectsApi()
```

```
apps_api = client.AppsV1Api()
core_api = client.CoreV1Api()

NAMESPACE = "default"

def deployment_exists(name):
    try:
        apps_api.read_namespaced_deployment(
            name=name,
            namespace=NAMESPACE
        )
        return True
    except ApiException as e:
        if e.status == 404:
            return False
        raise

def service_exists(name):
    try:
        core_api.read_namespaced_service(
            name=name,
            namespace=NAMESPACE
        )
        return True
    except ApiException as e:
        if e.status == 404:
            return False
        raise
```

```
def create_deployment(name, image, replicas, port):
```

```
    if deployment_exists(name):
```

```
        print(f"Deployment {name} already exists")
```

```
        return
```

```
    deployment = client.V1Deployment(
```

```
        metadata=client.V1ObjectMeta(
```

```
            name=name
```

```
        ),
```

```
        spec=client.V1DeploymentSpec(
```

```
            replicas=replicas,
```

```
            selector=client.V1LabelSelector(
```

```
                match_labels={
```

```
                    "app": name
```

```
                }
```

```
            ),
```

```
            template=client.V1PodTemplateSpec(
```

```
                metadata=client.V1ObjectMeta(
```

```
                    labels={
```

```
                        "app": name
```

```
                    }
```

```
                ),
```

```
                spec=client.V1PodSpec(
```

```
                    containers=[
```

```
                        client.V1Container(
```

```
                            name=name,
```



```
        image=image,

        ports=[
            client.V1ContainerPort(
                container_port=port
            )
        ]
    )
]
)
)
)
)
)
```

```
apps_api.create_namespaced_deployment(
    namespace=NAMESPACE,
    body=deployment
)
```

```
print(f"Deployment created: {name}")
```

```
def create_service(name, port):
```

```
    if service_exists(name):
        print(f"Service {name} already exists")
        return
```

```
    service = client.V1Service(
        metadata=client.V1ObjectMeta(
            name=name
        ),
```

```

spec=client.V1ServiceSpec(
    selector={
        "app": name
    },

    ports=[
        client.V1ServicePort(
            port=80,
            target_port=port
        )
    ],

    type="LoadBalancer"
)

core_api.create_namespaced_service(
    namespace=NAMESPACE,
    body=service
)

print(f"Service created: {name}")

def process_webapp(cr):

    name = cr["metadata"]["name"]

    spec = cr.get("spec", {})

    image = spec.get("image", "nginx")

```

```
replicas = spec.get("replicas", 1)
port = spec.get("port", 80)

print(
    f"Processing WebApplication: "
    f"{name}, image={image}, replicas={replicas}, port={port}"
)

create_deployment(
    name,
    image,
    replicas,
    port
)

create_service(
    name,
    port
)

while True:

    try:

        print("Watching WebApplication resources...")

        w = watch.Watch()

        for event in w.stream(
            crd_api.list_namespaced_custom_object,
            group="apps.company.com",
```

```
    version="v1",
    namespace=NAMESPACE,
    plural="webapplications",
    timeout_seconds=0
):

    event_type = event["type"]
    obj = event["object"]

    print(f"Event received: {event_type}")

    if event_type in ["ADDED", "MODIFIED"]:

        process_webapp(obj)

except Exception as e:

    print("Operator Error")
    traceback.print_exc()

    time.sleep(10)
```

Step 5: Create Dockerfile

Create:

```
Dockerfile
```

Paste:

```
FROM python:3.11-slim
WORKDIR /app
RUN pip install kubernetes
COPY webapp_operator.py .
CMD ["python", "webapp_operator.py"]
```

Step 6: Build Docker Image

```
docker build -t webapp-operator:v1 .
```

Step 7: Push Image to ECR

Create repository:

```
aws ecr create-repository --repository-name webapp-operator
```

Login:

```
aws ecr get-login-password \
| docker login \
--username AWS \
--password-stdin \
<ACCOUNT_ID>.dkr.ecr.<REGION>.amazonaws.com
```

```
docker tag webapp-operator:latest 962804699607.dkr.ecr.us-east-1.amazonaws.com/webapp-operator:latest
```

```
docker push 962804699607.dkr.ecr.us-east-1.amazonaws.com/webapp-operator:latest
```

Step 8: Deploy Operator

Create:

```
operator-deployment.yaml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: webapp-operator
spec:
  replicas: 1
  selector:
    matchLabels:
      app: webapp-operator
  template:
    metadata:
      labels:
        app: webapp-operator
    spec:
      serviceAccountName: webapp-operator
      containers:
        - name: operator
          image: 962804699607.dkr.ecr.us-east-1.amazonaws.com/webapp-operator:latest
```

Deploy:

```
kubectl apply -f operator-deployment.yaml
```

Verify:

```
kubectl get pods
```

Expected:

```
webapp-operator-xxxxx Running
```

Step 9: Create Custom Resource

Create:

```
ecommerce-app.yaml
```

Paste:

```
apiVersion: apps.company.com/v1
kind: WebApplication

metadata:
  name: ecommerce-app

spec:
  image: nginx
  replicas: 3
  port: 80
```

Deploy:

```
kubectl apply -f ecommerce-app.yaml
```

Step 11: Verify Deployment Creation

```
kubectl get deployments
```

Expected:

```
ecommerce-app
```

Verify Pods:

```
kubectl get pods
```

Expected:

```
ecommerce-app-xxxxx
```

```
ecommerce-app-yyyyy
```

```
ecommerce-app-zzzzz
```

Step 12: Verify Service Creation

```
kubectl get svc
```

Expected:

NAME	TYPE	EXTERNAL-IP
ecommerce-app	LoadBalancer	a1b2c3.elb.amazonaws.com

Step 13: Access NGINX

Open:

http://<LOADBALANCER-DNS>

Example:

```
http://a1b2c3.elb.amazonaws.com
```

Expected:

Welcome to nginx!

What Happened?

When you created:

```
kind: WebApplication
the Operator watched the CR:
WebApplication CR
  |
Operator
  |
  +---- Deployment
  +---- Service
The values:
image: nginx
replicas: 3
port: 80
```

were read by the Operator and converted into actual Kubernetes resources.

Learning Outcomes

After completing this lab, you can:

- Create a CRD
- Create Custom Resources
- Understand Kubernetes API extension
- Build a simple Operator

- Watch Custom Resources
- Implement a reconciliation workflow
- Automatically create Deployments and Services
- Access applications deployed from Custom Resources

This demonstrates the complete CRD → CR → Operator → Deployment → Service lifecycle that forms the